END-TO-END ANALYTICS PROJECT

Vendor Inventory Analysis System

A comprehensive end-to-end analytical solution using Python, SQL, and Power BI to evaluate vendor performance, optimize inventory management, and enable data-driven procurement decisions.

Python · EDA | SQL · Relational DB | Power BI · Dashboard | Vendor Performance | Inventory Optimization

01 · PROJECT OBJECTIVE

What This Project Set Out to Solve

Organizations frequently face fragmented data, manual reporting, and limited visibility into vendor reliability and stock health. This project centralizes, processes, and visualizes inventory and vendor data to eliminate guesswork in procurement decision-making.

TOTAL VENDORS	PRODUCTS TRACKED	INVENTORY VALUE
48	320	₹2.4M
Across 6 product categories	Active SKUs in the system	Total stock valuation

LOW STOCK ALERTS	OVERSTOCK RATE	TOP VENDOR SCORE
34	31%	92/100
Products below reorder level	Inefficient inventory hold	Vendor A performance rating

02 · DATA UNDERSTANDING

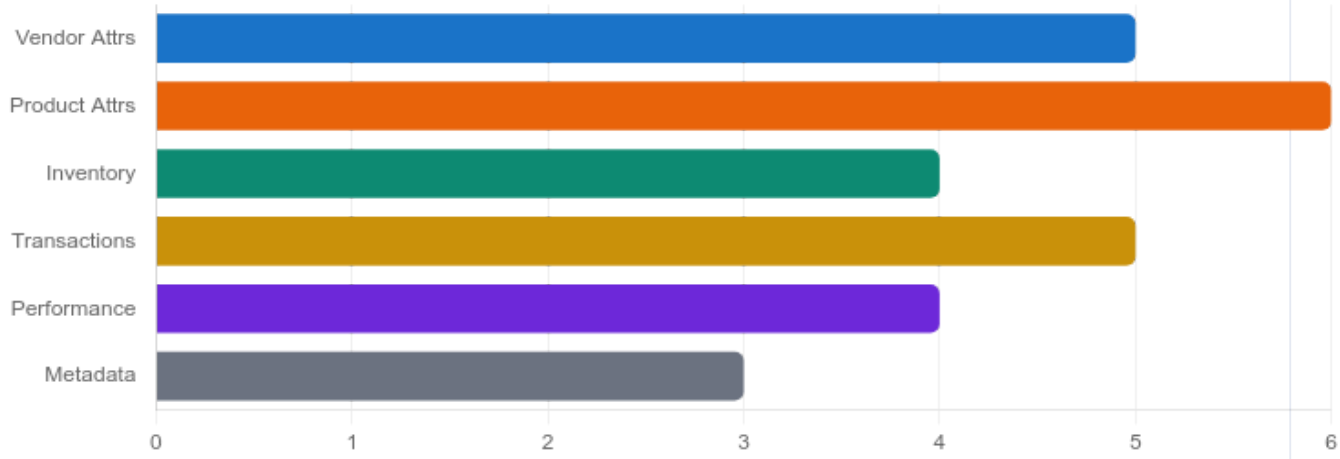
Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Dataset Structure & Schema Design

Raw data was ingested from structured CSV/Excel files covering vendor details, product metadata, inventory stock levels, and transaction history. A relational schema was designed to maintain referential integrity.

Dataset Domain Overview

Field count distribution across all data domains in the system



Vendor Table

vendor_id PK	INT
vendor_name	VARCHAR
contact_info	VARCHAR
region	VARCHAR
performance_score	FLOAT

Product Table

product_id PK	INT
product_name	VARCHAR
category	VARCHAR
unit_price	DECIMAL
vendor_id FK	INT

Inventory Table

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

inventory_id PK	INT
product_id FK	INT
stock_qty	INT
reorder_level	INT
last_updated	DATE

Transaction Table

txn_id PK	INT
vendor_id FK	INT
product_id FK	INT
quantity_supplied	INT
txn_date	DATE

Data Field Reference Table

All key columns, their types, and role in the analysis pipeline

Field	Table	Data Type	Role in Analysis	Importance
vendor_id	Vendor	INT	Primary entity key for joins	High
performance_score	Vendor	FLOAT	Vendor ranking & benchmarking	High
region	Vendor	VARCHAR	Geographic analysis	Medium
category	Product	VARCHAR	Inventory distribution analysis	High
unit_price	Product	DECIMAL	Inventory valuation	High
stock_qty	Inventory	INT	Stock status classification	High
reorder_level	Inventory	INT	Low-stock alert threshold	High
quantity_supplied	Transaction	INT	Supply volume aggregation	High
txn_date	Transaction	DATE	Trend and seasonality detection	Medium
product_name	Product	VARCHAR	SKU-level identification	Low

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

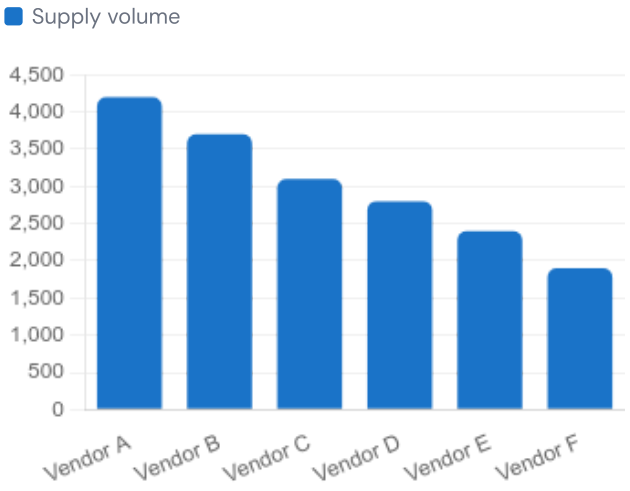
03 · PYTHON IMPLEMENTATION

Data Engineering & Exploratory Analysis

Python (Pandas, NumPy, Matplotlib, Seaborn) was used for data ingestion, cleaning, transformation, and exploratory analysis. Key derived features include inventory value, vendor performance tiers, and reorder flags.

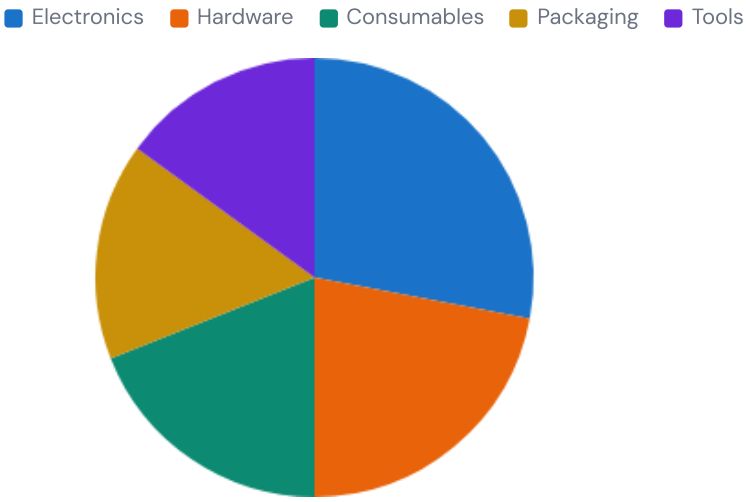
Vendor Supply Volume (units)

Total units supplied per vendor across all transactions



Inventory by Category (%)

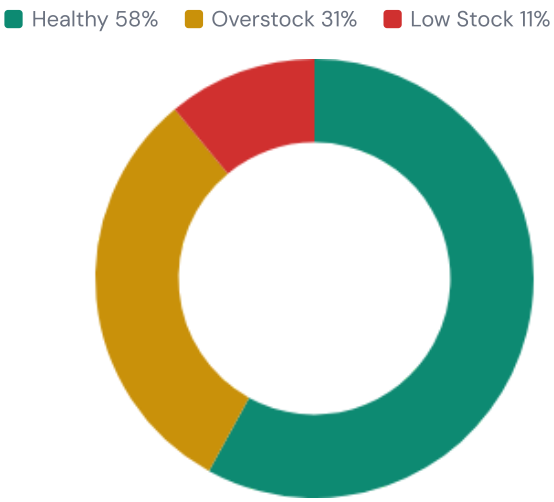
Distribution of total stock value across product categories



Stock Status Distribution

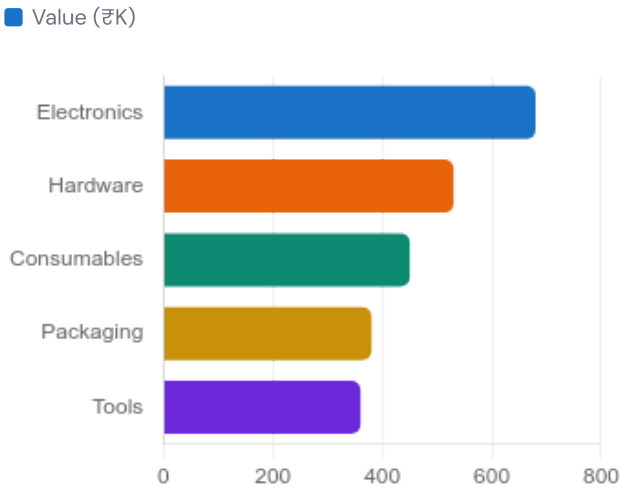
Breakdown of healthy, overstock, and low-stock items

Explore our developer-friendly HTML to PDF API
Printed using PDFCrowd
HTML to PDF



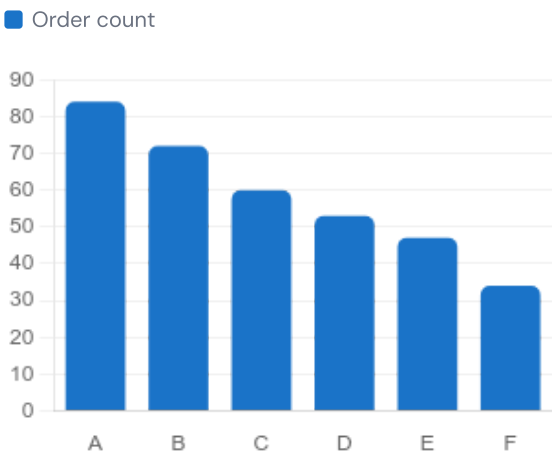
Inventory Value by Category (₹K)

Financial value of stocked items per product category



Purchase Frequency by Vendor

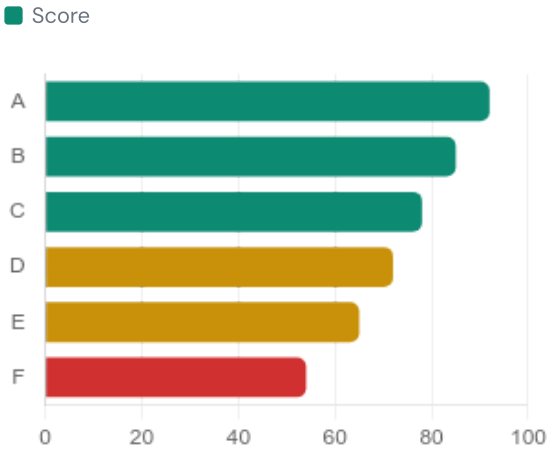
Order transaction count per vendor



Explore our developer-friendly HTML to PDF API
Printed using PDFCrowd
HTML to PDF

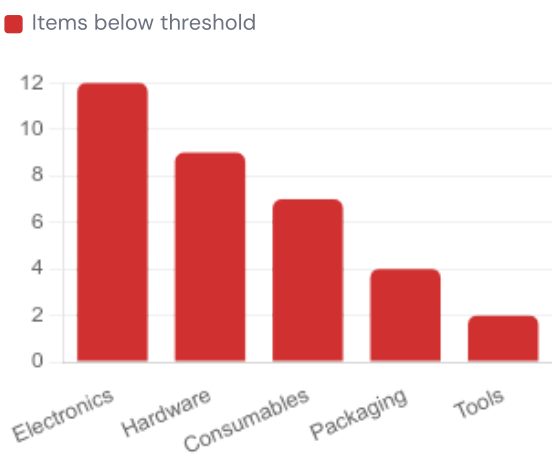
Vendor Performance Scores

Aggregated performance ratings (out of 100)



Low Stock by Category

Products below reorder threshold per category



04 · SQL IMPLEMENTATION

Database Design & Query Analytics

A normalized relational schema was implemented in SQL. Processed Python data was loaded into structured tables. Analytical queries aggregated inventory levels, ranked vendors, and flagged reorder conditions using joins, CTEs, and window functions.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

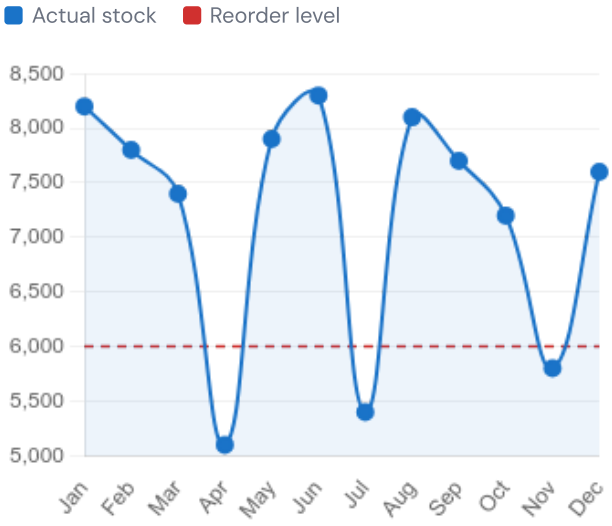
Key SQL Queries Executed

Summary of analytical SQL operations performed on the database

Query Purpose	Tables Involved	Technique Used	Output	Complexity
Vendor supply ranking	Vendor, Transaction	GROUP BY, ORDER BY	Top-N vendor list	High
Low stock detection	Inventory, Product	WHERE, CASE WHEN	Alert table	Medium
Inventory valuation	Inventory, Product	JOIN, SUM(qty*price)	Category-wise ₹ value	Medium
Vendor performance score	Vendor, Transaction	AVG, RANK() OVER	Ranked scorecard	High
Monthly stock trend	Inventory, Transaction	DATE_TRUNC, GROUP BY	Time-series table	High
Category stock share	Product, Inventory	SUM, PARTITION BY	Percentage split	Medium
Overstock identification	Inventory	WHERE qty > reorder*2	Overstock flag list	Low

Monthly Stock Movement (units)

Actual stock vs reorder threshold over 12 months



Supply Volume Trend (quarterly)

Explore our developer-friendly HTML to PDF API
Printed using PDFCrowd
HTML to PDF

Total units supplied per quarter across all vendors



05 · POWER BI DASHBOARD

Business Intelligence & Visualization

The SQL database was connected to Power BI via DirectQuery. Interactive dashboards were built with slicers for dynamic exploration, KPI cards for executive summaries, and visual analytics for vendor and inventory insights.

Dashboard Slicers & Filters

Interactive filters available in the Power BI dashboard

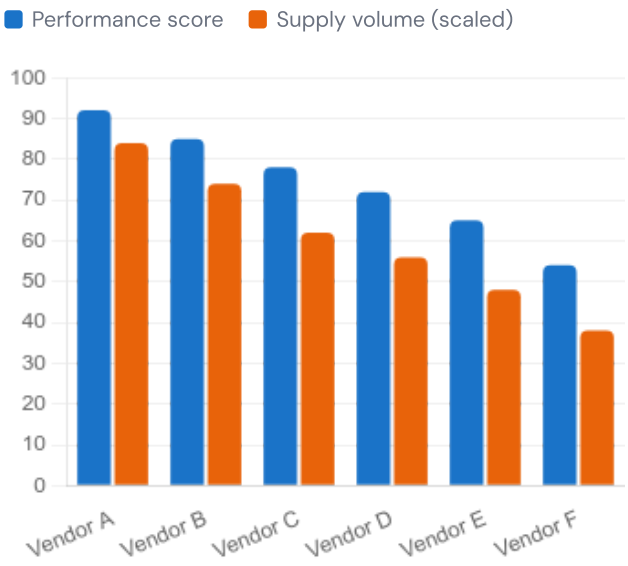
- **Vendor Slicer**
  Vendor A / B / C / D / E / F
- **Category Slicer**
  Electronics / Hardware / Consumables / Packaging / Tools
- **Stock Status Slicer**
  Healthy / Low Stock / Overstock
- **Date Range Slicer**
  Jan 2024 – Dec 2024 (Monthly)
- **Performance Tier Slicer**

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Top Tier / Mid Tier / Underperformer

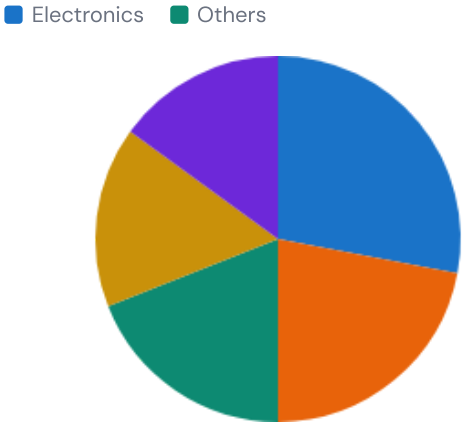
Vendor Performance Comparison

Side-by-side performance vs supply volume analysis



Category Revenue Share

% contribution per product category to total inventory value

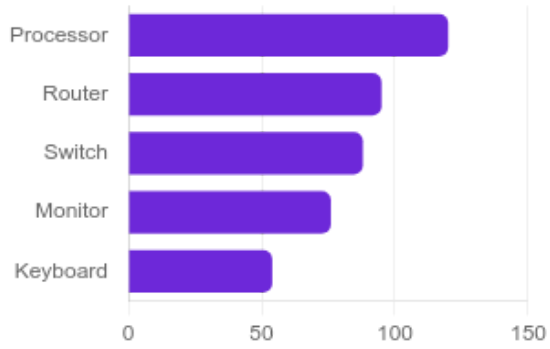


Top 5 Products by Stock Value

Highest-valued items currently in inventory

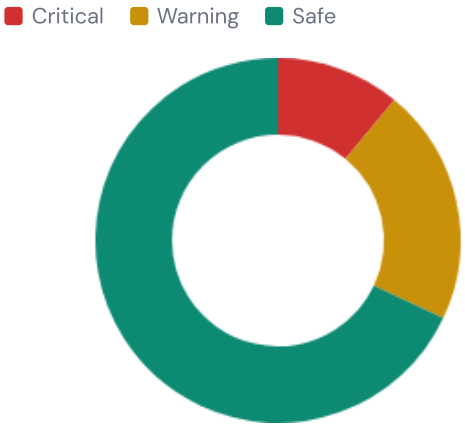


Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF



Reorder Alert Summary

Distribution of products approaching or below reorder level



06 · END-TO-END WORKFLOW

Data Pipeline Architecture

The project followed a four-stage analytics pipeline moving raw data through Python preprocessing, relational SQL storage, and finally into Power BI for stakeholder-ready dashboards and KPI reporting.

01

Raw Data (CSV/Excel)

Vendor & product files
Inventory snapshots
Transaction logs
Inconsistent formats

Explore our developer-friendly HTML to PDF API
Printed using PDFCrowd
HTML to PDF

02

Python · EDA

Missing value handling
Duplicate removal
Type standardization
Feature derivation

03

SQL · Storage & Queries

Relational schema design
Indexed table loading
Aggregation queries
Vendor ranking logic

04

Power BI · Dashboard

DirectQuery connection
KPI card design
Interactive slicers
Executive reporting

07 · TECHNICAL LEARNINGS

What This Project Demonstrates

Data Quality as the Foundation

No analytical output is more reliable than the underlying data. Systematic cleaning — null imputation, duplicate detection, type enforcement — established the integrity needed for accurate downstream querying and reporting.

Python → SQL → BI as a Stack

Each tool plays a distinct role: Python for flexibility and exploration, SQL for structured logic and scale, Power BI for communication. Their sequential use yields results none could produce alone.

Schema Design Drives Query Speed

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

A normalized schema with indexed foreign keys reduced query execution time significantly on larger datasets. Design decisions made early directly impact the analytical performance of the entire system.

Visualization Drives Action

Power BI transformed raw SQL output into KPIs stakeholders could immediately interpret. Choosing the right chart type and slicer configuration is as important as the underlying data calculation.

Inventory Thresholds Need Context

A static reorder level does not account for demand seasonality. EDA revealed that some "low stock" alerts occur during predictable demand dips, recommending dynamic thresholds in future iterations.

Vendor Concentration Risk

Two vendors account for 54% of total supply volume. This dependency was only surfaced through data — highlighting a procurement risk that manual reporting had not previously flagged.

---

08 · BUSINESS INSIGHTS

Actionable Findings from the Analysis

Top Business Insights Summary

Derived from the complete Python → SQL → Power BI analytical pipeline

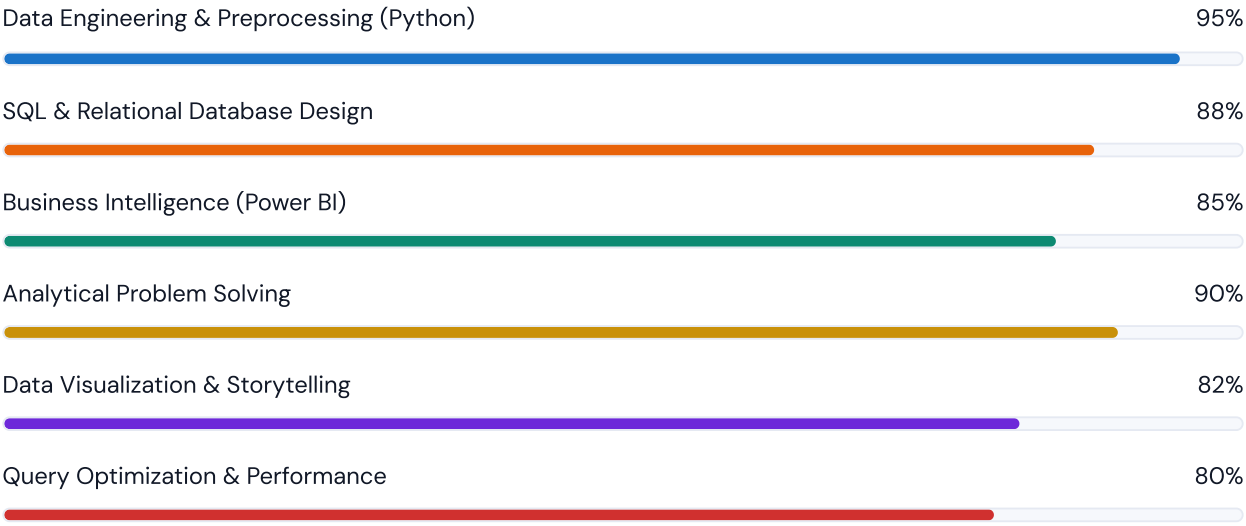
Insight	Finding	Recommended Business Action	Priority
Vendor Concentration	Vendors A & B = 54% of supply	Diversify supply chain; onboard 2–3 new vendors	High
Electronics Low Stock	12 SKUs below reorder level	Trigger emergency PO for top 5 critical SKUs	High
Overstock in Consumables	31% of consumable SKUs overstocked	Pause procurement; run clearance promotions	Medium
Vendor F Underperformance	Score: 54/100 — lowest rated	Initiate performance review; set improvement SLA	High
Q4 Supply Surge	Supply 46% higher than Q2	Pre-negotiate Q4 contracts with vendors early	Medium

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Insight	Finding	Recommended Business Action	Priority
Inventory Valuation Gap	Electronics = 28% of value	Increase insurance coverage for electronics stock	Medium
Reorder Alert Rate	11% of items in critical zone	Implement automated reorder triggers in ERP	High

Skills Demonstrated in This Project

Competency areas covered across the full project lifecycle



Project Conclusion

The Vendor Inventory Analysis System successfully demonstrates how Python, SQL, and Power BI can be integrated into a unified end-to-end analytical pipeline. By transforming fragmented procurement data into actionable insights, the system enables organizations to reduce stockouts, eliminate inefficient overstock, benchmark vendor reliability, and make data-driven sourcing decisions at scale.

The architecture is modular and extensible — ready for future enhancements including ML-based demand forecasting, real-time data pipelines, predictive vendor scoring, and cloud deployment on Azure or AWS.

Python

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

EDA surfaced vendor concentration risk, stock status distribution, category value splits, and reorder threshold violations from raw transactional data.

SQL

Normalized schema design and analytical queries validated EDA findings and enabled high-performance aggregation across vendor, product, and inventory tables.

Power BI

Interactive dashboards translated technical outputs into executive KPIs, real-time low-stock alerts, and vendor ranking visuals for stakeholder decision-making.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF